

LAPORAN TUGAS 2

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mohamad Qowi Aizani (2410651003)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

TUJUAN

- Mempelajari dan mengimplementasikan cara kerja *process management* pada sistem operasi dengan membuat beberapa proses anak yang menjalankan tugas berbeda, serta mengukur waktu eksekusi dan status masing-masing proses.
- Mempelajari *Inter Process Communication (IPC)* melalui implementasi aplikasi chat sederhana menggunakan *Shared Memory* dan *Semaphore* untuk sinkronisasi antar proses.

Dasar Teori

a. Proses dan Manajemen Proses

Proses adalah program yang sedang dieksekusi. Setiap proses memiliki *Process ID (PID)*, ruang memori, dan status tersendiri.

Sistem operasi bertanggung jawab dalam membuat (*create*), menjalankan (*execute*), dan menghentikan (*terminate*) proses.

Konsep *process management* mencakup pembuatan proses anak (*fork*), komunikasi antar proses, serta sinkronisasi waktu eksekusi.

b. Inter Process Communication (IPC)

IPC adalah mekanisme yang digunakan oleh proses-proses dalam sistem operasi untuk bertukar data atau berkoordinasi.

Metode umum IPC antara lain:

- Pipes / Named Pipes
- Message Queue
- Shared Memory
- Sockets
- Semaphores & Mutex

Pada praktikum ini digunakan **Shared Memory** untuk berbagi data antar proses dan **Semaphore** sebagai alat sinkronisasi agar tidak terjadi konflik saat beberapa proses mengakses memori yang sama.

Tugas 1

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <sys/time.h>

#include <string.h>

#include <time.h>

#include <errno.h>


#define HASIL_BUF 256


struct pesan_anak {

    pid_t pid;

    long waktu_eksekusi;

    int kode_status;

    char nama_tugas[32];

    char hasil[HASIL_BUF];

};


long selisih_waktu_ms(struct timeval *mulai, struct timeval *selesai) {

    long detik = selesai->tv_sec - mulai->tv_sec;

    long mikro = selesai->tv_usec - mulai->tv_usec;

    return detik * 1000 + mikro / 1000;
```

```
}
```

```
int banding_int(const void *a, const void *b) {
```

```
    int ia = *(const int*)a;
```

```
    int ib = *(const int*)b;
```

```
    return (ia > ib) - (ia < ib);
```

```
}
```

```
void tugas_sorting(char *keluaran, size_t ukuran) {
```

```
    const int N = 10000;
```

```
    int *data = malloc(sizeof(int)*N);
```

```
    if (!data) { snprintf(keluaran, ukuran, "gagal malloc"); return; }
```

```
    for (int i=0;i<N;i++) data[i] = rand() % 1000000;
```

```
    qsort(data, N, sizeof(int), banding_int);
```

```
    int median = data[N/2];
```

```
    int batas = 5;
```

```
    char temp[HASIL_BUF];
```

```
    int pos = snprintf(temp, sizeof(temp), "median=%d; awal=%d", median, data[0]);
```

```
    for (int i=1;i<batas && pos < (int)sizeof(temp)-50;i++) {
```

```
        pos += snprintf(temp+pos, sizeof(temp)-pos, ",%d", data[i]);
```

```
    }
```

```
    snprintf(keluaran, ukuran, "%s", temp);
```

```
    free(data);
```

```
}
```

```
void tugas_pencarian(char *keluaran, size_t ukuran) {
```

```

const int N = 5000;

int *data = malloc(sizeof(int)*N);

if (!data) { snprintf(keluaran, ukuran, "gagal malloc"); return; }

for (int i=0;i<N;i++) data[i] = i*2;

int kunci = rand() % (N*2);

int kiri=0, kanan=N-1, ditemukan = -1;

while (kiri<=kanan) {

    int tengah = (kiri+kanan)/2;

    if (data[tengah] == kunci) { ditemukan = tengah; break; }

    else if (data[tengah] < kunci) kiri = tengah+1;

    else kanan = tengah-1;

}

if (ditemukan >= 0)

    snprintf(keluaran, ukuran, "kunci=%d DITEMUKAN di indeks=%d", kunci, ditemukan);

else

    snprintf(keluaran, ukuran, "kunci=%d TIDAK DITEMUKAN", kunci);

free(data);

}

```

```

void tugas_perhitungan(char *keluaran, size_t ukuran) {

    const int BATAS = 100000;

    char *prima = calloc(BATAS+1, sizeof(char));

    if (!prima) { snprintf(keluaran, ukuran, "gagal calloc"); return; }

    for (int i=2;i<=BATAS;i++) prima[i] = 1;

    for (int p=2;p*p<=BATAS;p++) {

        if (prima[p]) {

```

```

        for (int q=p*p;q<=BATAS;q+=p) prima[q]=0;
    }
}

long long jumlah = 0;

int hitung = 0;

for (int i=2;i<=BATAS;i++) if (prima[i]) { jumlah += i; hitung++; }

snprintf(keluaran, ukuran, "jumlah_prima<=%d = %lld (banyak=%d)", BATAS, jumlah,
        hitung);

free(prima);
}

```

```

void proses_anak(int fd_tulis, const char *tugas) {

    struct pesan_anak pesan;

    memset(&pesan, 0, sizeof(pesan));

    pesan.pid = getpid();

    strncpy(pesan.nama_tugas, tugas, sizeof(pesan.nama_tugas)-1);

    struct timeval t0, t1;

    gettimeofday(&t0, NULL);

    srand(((unsigned int)(time(NULL) ^ (pesan.pid<<16))));

    if (strcmp(tugas, "sorting") == 0) {

        tugas_sorting(pesan.hasil, sizeof(pesan.hasil));

        pesan.kode_status = 0;

    } else if (strcmp(tugas, "pencarian") == 0) {

        tugas_pencarian(pesan.hasil, sizeof(pesan.hasil));

        pesan.kode_status = 0;

    } else if (strcmp(tugas, "perhitungan") == 0) {

        tugas_perhitungan(pesan.hasil, sizeof(pesan.hasil));
    }
}

```

```

    pesan.kode_status = 0;
} else {
    snprintf(pesan.hasil, sizeof(pesan.hasil), "tugas tidak dikenal");
    pesan.kode_status = 1;
}

gettimeofday(&t1, NULL);
pesan.waktu_eksekusi = selisih_waktu_ms(&t0, &t1);
write(fd_tulis, &pesan, sizeof(pesan));
close(fd_tulis);
_exit(pesan.kode_status);
}

int main(void) {
    const char *daftar_tugas[] = {"sorting", "pencarian", "perhitungan"};
    const int JUMLAH_TUGAS = sizeof(daftar_tugas)/sizeof(daftar_tugas[0]);
    int pipa[JUMLAH_TUGAS][2];
    pid_t pid_anak[JUMLAH_TUGAS];
    for (int i=0;i<JUMLAH_TUGAS;i++) {
        if (pipe(pipa[i]) == -1) {
            perror("gagal membuat pipa");
            exit(EXIT_FAILURE);
        }
        pid_t pid = fork();
        if (pid < 0) {
            perror("gagal fork");
            exit(EXIT_FAILURE);
        }
    }
}

```



```

    } else if (pid == 0) {

        close(pipa[i][0]);

        proses_anak(pipa[i][1], daftar_tugas[i]);

    } else {

        pid_anak[i] = pid;

        close(pipa[i][1]);

    }

}

struct pesan_anak pesan[JUMLAH_TUGAS];

memset(pesan, 0, sizeof(pesan));

for (int i=0;i<JUMLAH_TUGAS;i++) {

    ssize_t r = read(pipa[i][0], &pesan[i], sizeof(pesan[i]));

    if (r <= 0) {

        snprintf(pesan[i].hasil, sizeof(pesan[i].hasil), "tidak ada pesan (r=%zd, errno=%d)", r,
        errno);

        pesan[i].pid = pid_anak[i];

        pesan[i].waktu_eksekusi = -1;

        pesan[i].kode_status = -1;

        strncpy(pesan[i].nama_tugas, daftar_tugas[i], sizeof(pesan[i].nama_tugas)-1);

    }

    close(pipa[i][0]);

}

printf("=== LAPORAN MANAJER PROSES ===\n");

for (int i=0;i<JUMLAH_TUGAS;i++) {

    int status;

    pid_t w = waitpid(pid_anak[i], &status, 0);

    if (w == -1) perror("waitpid");

```

```

char teks_status[128];

if (WIFEXITED(status)) {

    int kode = WEXITSTATUS(status);

    snprintf(teks_status, sizeof(teks_status), "keluar normal, kode=%d", kode);

} else if (WIFSIGNALED(status)) {

    int sinyal = WTERMSIG(status);

    snprintf(teks_status, sizeof(teks_status), "dihentikan oleh sinyal %d", sinyal);

} else {

    snprintf(teks_status, sizeof(teks_status), "terminasi tidak diketahui");

}

pid_t pid_lapor = pesan[i].pid ? pesan[i].pid : pid_anak[i];

printf("\nTugas: %s\n", pesan[i].nama_tugas);

printf(" PID          : %d\n", (int)pid_lapor);

if (pesan[i].waktu_eksekusi >= 0)

    printf(" Waktu eksekusi : %ld ms\n", pesan[i].waktu_eksekusi);

else

    printf(" Waktu eksekusi : (tidak dilaporkan)\n");

printf(" Status (wait)  : %s\n", teks_status);

printf(" Status anak    : %d\n", pesan[i].kode_status);

printf(" Hasil ringkas   : %s\n", pesan[i].hasil);

}

printf("\nSemua proses anak selesai. Proses induk keluar.\n");

return 0;

}

```

```

qowi@QOWIAIZANI:~$ gcc -o proc_manager proc_manager.c
qowi@QOWIAIZANI:~$ ./proc_manager
=== LAPORAN MANAJER PROSES ===

Tugas: sorting
  PID      : 34995
  Waktu eksekusi : 1 ms
  Status (wait) : keluar normal, kode=0
  Status anak   : 0
  Hasil ringkas : median=502077; awal=0,119,300,424,490

Tugas: pencarian
  PID      : 34996
  Waktu eksekusi : 0 ms
  Status (wait) : keluar normal, kode=0
  Status anak   : 0
  Hasil ringkas : kunci=1963 TIDAK DITEMUKAN

Tugas: perhitungan
  PID      : 34997
  Waktu eksekusi : 1 ms
  Status (wait) : keluar normal, kode=0
  Status anak   : 0
  Hasil ringkas : jumlah_prima<=100000 = 454396537 (banyak=9592)

Semua proses anak selesai. Proses induk keluar.
qowi@QOWIAIZANI:~$ |

```

Analisis Hasil

- Program berhasil membuat **3 proses anak**, masing-masing menjalankan tugas berbeda: sorting, searching, dan perhitungan.
- Semua proses memiliki **status keluar normal (kode=0)**, artinya tidak ada error.
- **Waktu eksekusi cepat (0–1 ms)** menandakan efisiensi program.
- Output menampilkan **PID unik** untuk tiap proses anak.
- Parent menunggu semua anak selesai sebelum keluar → **sinkronisasi berhasil**.

TUGAS 2

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#include <semaphore.h>
#include <errno.h>

#define SHM_NAME "/chat_shm_example_01"
#define SEM_NAME "/chat_sem_example_01"
#define MAX_PESAN 100
#define MAX_USER 32
#define MAX_TKST 256

struct pesan {
    time_t waktu;
    char pengguna[MAX_USER];
    char teks[MAX_TKST];
};

struct ruang_obrolan {
    int kepala;
    int jumlah;
    struct pesan daftar[MAX_PESAN];
};

static void format_waktu(time_t t, char *out, size_t n) {
    struct tm tm;
    localtime_r(&t, &tm);
    strftime(out, n, "%Y-%m-%d %H:%M:%S", &tm);
}

static int buat_ukuran_shm(int fd, size_t size) {
    if (ftruncate(fd, size) == -1) {
        perror("ftruncate");
        return -1;
    }
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Penggunaan: %s <NamaUser> | --bersih\n", argv[0]);
        return 1;
    }
}
```

```

}

if (strcmp(argv[1], "--bersih") == 0) {
    if (sem_unlink(SEM_NAME) == -1 && errno != ENOENT) perror("sem_unlink");
    else printf("Semaphore %s dihapus (jika ada)\n", SEM_NAME);
    if (shm_unlink(SHM_NAME) == -1 && errno != ENOENT) perror("shm_unlink");
    else printf("Shared memory %s dihapus (jika ada)\n", SHM_NAME);
    return 0;
}

char nama_pengguna[MAX_USER];
strncpy(nama_pengguna, argv[1], MAX_USER - 1);
nama_pengguna[MAX_USER - 1] = '\0';

int baru = 0;
int shm_fd = shm_open(SHM_NAME, O_RDWR | O_CREAT | O_EXCL, 0666);
if (shm_fd >= 0) {
    baru = 1;
    size_t ukuran = sizeof(struct ruang_obrolan);
    if (buat_ukuran_shm(shm_fd, ukuran) == -1) {
        close(shm_fd);
        return 1;
    }
} else if (errno == EEXIST) {
    shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        return 1;
    }
} else {
    perror("shm_open");
    return 1;
}

size_t map_size = sizeof(struct ruang_obrolan);
struct ruang_obrolan *ruang = mmap(NULL, map_size, PROT_READ | PROT_WRITE,
MAP_SHARED, shm_fd, 0);
if (ruang == MAP_FAILED) {
    perror("mmap");
    close(shm_fd);
    return 1;
}

sem_t *sem = NULL;
if (baru) {
    sem = sem_open(SEM_NAME, O_CREAT | O_EXCL, 0666, 1);
    if (sem == SEM_FAILED) {
        perror("sem_open");
        munmap(ruang, map_size);
        close(shm_fd);
        shm_unlink(SHM_NAME);
        return 1;
    }
}

```

```

    }
    sem_wait(sem);
    ruang->kepala = 0;
    ruang->jumlah = 0;
    sem_post(sem);
} else {
    sem = sem_open(SEM_NAME, 0);
    if (sem == SEM_FAILED) {
        perror("sem_open");
        munmap(ruang, map_size);
        close(shm_fd);
        return 1;
    }
}
}

```

```

printf("=== RUANG OBROLAN ===\n");
printf("Pengguna: %s\n", nama_pengguna);
printf("Perintah: 'kirim', 'lihat', 'keluar'\n");

```

```

char baris[512];
while (1) {
    printf("%s> ", nama_pengguna);
    fflush(stdout);
    if (!fgets(baris, sizeof(baris), stdin)) {
        printf("\nKeluar...\n");
        break;
    }
    size_t L = strlen(baris);
    if (L > 0 && baris[L - 1] == '\n') baris[L - 1] = '\0';

    if (strcmp(baris, "kirim") == 0) {
        printf("Tulis pesan: ");
        fflush(stdout);
        char teks[MAX_TEKST];
        if (!fgets(teks, sizeof(teks), stdin)) continue;
        size_t tl = strlen(teks);
        if (tl > 0 && teks[tl - 1] == '\n') teks[tl - 1] = '\0';
        sem_wait(sem);
        int idx = ruang->kepala;
        ruang->daftar[idx].waktu = time(NULL);
        strncpy(ruang->daftar[idx].pengguna, nama_pengguna, MAX_USER - 1);
        strncpy(ruang->daftar[idx].teks, teks, MAX_TEKST - 1);
        ruang->kepala = (ruang->kepala + 1) % MAX_PESAN;
        if (ruang->jumlah < MAX_PESAN) ruang->jumlah++;
        sem_post(sem);
    } else if (strcmp(baris, "lihat") == 0) {
        sem_wait(sem);
        int jumlah = ruang->jumlah;
        if (jumlah == 0) printf("(Belum ada pesan)\n");
        else {
            int mulai = (ruang->kepala - jumlah + MAX_PESAN) % MAX_PESAN;
            for (int i = 0; i < jumlah; ++i) {

```

```

        int idx = (mulai + i) % MAX_PESAN;
        char waktu_str[64];
        format_waktu(ruang->daftar[idx].waktu, waktu_str, sizeof(waktu_str));
        printf("[%s] %s> %s\n", waktu_str, ruang->daftar[idx].pengguna, ruang-
>daftar[idx].teks);
    }
}
sem_post(sem);
} else if (strcmp(baris, "keluar") == 0) {
    printf("Keluar dari chat.\n");
    break;
} else {
    printf("Perintah tidak dikenal.\n");
}
}

munmap(ruang, map_size);
close(shm_fd);
sem_close(sem);
return 0;
}

```

Analisis:

```
qowi@QOWIAIZANI:~$ nano chat_shm.c
qowi@QOWIAIZANI:~$ gcc -o chat_shm chat_shm.c -pthread -lrt
qowi@QOWIAIZANI:~$ ./chat_shm User1
=== RUANG OBROLAN ===
Pengguna: User1
Perintah: ' kirim', ' lihat', ' keluar'
User1> kirim
Tulis pesan: Saya seorang mahasiswa di Universitas Muhammadiyah Jember
User1> keluar
Keluar dari chat.
qowi@QOWIAIZANI:~$ |
```

- User1 masuk ke ruang obrolan.
- Program memberikan 3 perintah utama:
kirim, **lihat**, dan **keluar**.
- User1 memilih kirim, lalu menulis pesan:
- “Saya seorang mahasiswa di Universitas Muhammadiyah Jember”
- Pesan disimpan ke dalam **shared memory**.
- Setelah itu User1 mengetik keluar, dan program menampilkan “Keluar dari chat.”
→ menandakan sesi chat User1 selesai dengan benar.

```
qowi@QOWIAIZANI:~$ ./chat_shm User2
=== RUANG OBROLAN ===
Pengguna: User2
Perintah: ' kirim', ' lihat', ' keluar'
User2> lihat
[2025-10-22 10:21:17] User1> Saya seorang mahasiswa di Universitas Muhammadi
yah Jember
User2> |
```

- User2 juga masuk ke ruang obrolan.
- Ketika User2 mengetik perintah lihat, program membaca data dari **shared memory**.
- Terlihat pesan dari User1:
- [2025-10-22 10:21:17] User1> Saya seorang mahasiswa di Universitas Muhammadiyah Jember
- → Artinya pesan dari User1 berhasil **tersimpan dan dapat diakses antar proses**.
- Format pesan juga memuat **timestamp**, nama pengguna, dan isi pesan, menandakan program mengatur log chat dengan baik.

KESIMPULAN

- Sistem operasi mengatur pembuatan, eksekusi, dan sinkronisasi proses agar berjalan efisien.
- IPC memungkinkan proses saling bertukar data tanpa konflik.
- Shared memory memberi performa tinggi, sedangkan semaphore menjaga konsistensi data.
- Perbandingan metode IPC:
 - **Shared Memory:** sangat cepat, tetapi hanya lokal.
 - **Message Queue:** lebih fleksibel, cocok untuk sistem terdistribusi.
 - **Pipe:** mudah diimplementasikan tapi satu arah.
 - **Socket:** cocok untuk jaringan, tapi kompleks.
- Melalui praktikum ini, mahasiswa memahami konsep *process management* dan *IPC synchronization* secara nyata melalui implementasi C di Linux.